BCB744: Intro R test

A. J. Smit
University of the Western Cape

2025-03-17

About the test

The Intro R Test will start at 8:30 on 17 March, 2025 and you have until 08:30 on 18 March to complete it. The Theory Test must be conducted on campus, and the Practical Test at home or anywhere you are comfortable working. The test constitutes a key component of Continuous Assessment (CA) and are designed to prepare you for the final exam.

The test consists of two parts:

Theory test (30%)

This is a written, closed-book assessment where you will be tested on theoretical concepts. The only resource available during this test is RStudio, the R help system, your memory, and your mind.

Practical test (70%)

In this open-book coding assessment, you will apply your theoretical knowledge to real data problems. While you may reference online materials (including ChatGPT), collaboration with peers is strictly prohibited.

Assessment policy

The marks indicated for each section reflect the relative weight (and hence depth expected in your response) rather than a rigid checklist of individual points. Your answer should demonstrate a comprehensive understanding of the concepts and techniques required, showing thoughtful integration of multiple R skills. Higher marks will be awarded for solutions that demonstrate not only technical correctness but also elegant code, insightful analysis, and clear communication of findings. We are assessing your ability to think systematically through complex data problems, make appropriate methodological choices, and present your findings in a coherent narrative that reveals meaningful patterns in the data. Your code should be well-structured, adequately commented, and reflect good programming practices.

Please refer to the Assessment Policy for more information on the test format and rules.

Theory test

This is the closed book assessment.

Below is a set of questions to answer. You must answer all questions in the allocated time of 3-hr. Please write your answers in a neatly formatted Word document and submit it to the iKamva platform.

Clearly indicate the question number and provide detailed explanations for your answers. Use Word's headings and subheadings facility to structure your document logically.

Naming convention: `Intro_R_Test_Theory_YourSurname.docx`

Question 1

You are a research assistant who have just been given your first job. You are asked to analyse a dataset about patterns of extreme heat in the ocean and the possible role that ocean currents (specifically, eddies) might play in modulating the patterns of extreme sea surface temperature extremes in space and time.

Being naive and relatively inexperienced, and misguided by your exaggerated sense of preparedness as young people tend to do, you gladly accept the task and start by exploring the data. You notice that the dataset is quite large, and you have no idea what's happening, what you are doing, why you are doing it, or what you are looking for. Ten minutes into the job you start to question your life choices. Your feeling of bewilderment is compounded by the fact that, when you examine the data (the output of the `head()` and `tail()` commands is shown below), the entries seem confusing.

```
fpath <- "/Volumes/OceanData/spatial/processed/WBC/misc_results"
fname <- "KC-MCA-data-2013-01-01-2022-12-31-bbox-v1_ma_14day_detrended.csv"
data <- read.csv(file.path(fpath, fname))
```

```
> nrow(data)
[1] 53253434

> head(data)
           t     lon    lat      ex    ke
1 2013-01-01 121.875 34.625 -0.7141 2e-04
2 2013-01-01 121.875 34.625 -0.8027 2e-04
3 2013-01-02 121.875 34.625 -0.8916 2e-04
4 2013-01-02 121.875 34.625 -0.9751 2e-04
5 2013-01-03 121.875 34.625 -1.0589 3e-04
6 2013-01-03 121.875 34.625 -1.1406 3e-04

> tail(data)
                   t     lon    lat     ex      ke
53253429 2022-12-29 174.375 44.875 0.4742 -0.0049
53253430 2022-12-29 174.375 44.875 0.4856 -0.0049
53253431 2022-12-30 174.375 44.875 0.4969 -0.0050
```

```
53253432 2022-12-30 174.375 44.875 0.5169 -0.0050
53253433 2022-12-31 174.375 44.875 0.5367 -0.0051
53253434 2022-12-31 174.375 44.875 0.5465 -0.0051
```

You resign yourself to admitting that you don't understand much, but at the risk of sounding like a fool when you go to your professor, you decide to do as much of the preparation you can do so that you at least have something to show for your time.

a. What will you take back to your professor to show that you have prepared yourself as fully as possible? For example:
   - What is in your ability to understand about the study and the nature of the data?
   - What will you do for yourself to better understand the task at hand?
   - What do you understand about the data?
   - What will you do to aid your understanding of the data?
   - What will your next steps be going forward?

b. What will you need from your professor to help you understand the data and the task at hand so that you are well equipped to tackle the problem?

[15 marks]

Answer

- I am able to understand what the concept of 'extreme heat' is, and what ocean eddies are – all I need to do is find some papers about it and do broad reading around these concepts. So, I will start by reading up on these concepts.
- I can see from the columns that there appears to be three independent variables (`lon`, `lat`, and `t`) and two dependent variables (`ex` and `ke`). I will need to understand what these variables are, and how they relate to each other. It is easy to see that `lon` and `lat` are the longitude and latitude of the data points, and that `t` is the date of the data point. I will need to understand what the `ex` and `ke` variables are, and how they relate to the `lon` and `lat` variables. Presumably `ex` and `ke` are the extreme heat and ocean eddies, respectively. I'll confirm with the professor.
- Because I have `lon` and `lat`, I can make a map of the study area. By making a map of the study area for one or a few days in the dataset, I can get a sense of the spatial distribution of the data. I can also plot the `ex` and `ke` data to see what the data look like. Because the data cover the period 2013-2022, I know that I can create a map for each day (a time-series analysis might eventually be needed?), and that is probably where the analysis will takle me later once I have confirmed my thinking with the professor. If I am really proactive and want to seriously impress the professor, I'll make an animation of the data to show the temporal evolution of revealed patterns in the data over time. This will clearly show the processes operating there. A REALLY informed mind will be able to even go as far as understanding what the analysis shoud entail, but, admittedly, this will require a deep subject matter understanding, which you might not possess at the moment, but which is nevertheless not beyond your reach to attain without guidance.

- I can conclude that the data reveal some dynamical process (I infer 'dynamical' from the fact that we have time-series data, and time-series reveal dynamics).
- Knowing what the geographical region is from the map I created and what is happening there that might be of interest to the study, I can make some guesses about what the analysis will be.
- FYI, what basic reseach would reveal include the following (not for marks):
  - you'd see that it is an ocean region south of South Africa;
  - once you know the region covered, you can read about the processes operating in the region that the data cover;
  - because the temperature spatially defines the Agulhas Current, you can infer that the study is about the Agulhas Current
  - plotting `ke` will reveal eddies in the Agulhas Current;
  - you can read about the Agulhas Current and its eddies and think about how eddies might affect the temperature in the region – both of these are dynamical processes.
- I will need to understand what the data are telling me, and what the variables mean. I will need to understand what the `ex` and `ke` variables are, and how they relate to the `lon` and `lat` variables.
- Having discovered all these things simply by doing a basic first-stab analyses, I can prepare a report of my cursory findings and draw of a list of things I know, toghether with suggested further avenues for exploration. I will take this to the professor to confirm my understanding and to get guidance on how to proceed.
- I will also add a list of the things I cannot know from the data, and what I need to know from the professor to proceed.
- There is also something strange happening with the data. It seems that there are duplicate data entries (two occurrences of each combination of `lat` x `lon` x `t` resulting in duplicated values for each spatio-temporal point of `ke` and a pair of dissimilar values for `ex`). I will need to understand why this is the case. Clearly this is incorrect, and this points to pre-processing errors somewhere. I will have to ask the professor to give me access to all pro-processing scripts and the raw data to see if I can trace the error back to its source.
- If I was this professor, I'd be immensepy mpressed by tyour proactive approach to the problem. You are showing that you are not just a passive learner, but that you are actively engaging with the data and the problem at hand. This is a very good sign of a good researcher in the making. In my mind, I'd seriously think about finding you a salary for permanent employment in my lab.

Question 2

Please translate the following code into English by providing an explanation for each line:

```
monthlyData <- dailyData %>%
    dplyr::mutate(t = asPOSIXct(t)) %>%
    dplyr::mutate(month = floor_date(t, unit = "month")) %>%
    dplyr::group_by(lon, lat, month) %>%
    dplyr::summarise(temp = mean(temp, na.rm = TRUE)) %>%
    dplyr::mutate(year = year(month)) %>%
    dplyr::group_by(lon, lat) %>%
```

```
    dplyr::mutate(num = seq(1:length(temp))) %>%
    dplyr::ungroup()
```

In your answer, simply refer to the line numbers (1-9) before each line of code and provide an explanation for each line.

[10 marks]

Answer

- Line 1: The variable `monthlyData` is created by starting with `dailyData`, which is a dataset containing daily records.
- Line 2: The `mutate()` function is used to convert the column `t` (presumably a date or timestamp) into a POSIXct datetime format. This ensures that `t` is stored in a standardised date-time format suitable for time-based operations.
- Line 3: The `mutate()` function is again used to create a new column `month`, which is derived from `t`. The `floor_date()` function rounds down the date to the first day of the corresponding month, effectively extracting the month from `t`.
- Line 4: The `group_by()` function groups the dataset by `lon` (longitude), `lat` (latitude), and `month`. This means subsequent operations will be performed separately for each unique combination of these three variables.
- Line 5: The `summarise()` function computes the mean temperature (`temp`) for each group. The `na.rm = TRUE` argument ensures that missing values (`NA`) are ignored in the calculation.
- Line 6: The `mutate()` function creates a new column, `year`, extracting the year from the `month` column. This provides an explicit reference to the year of each data entry.
- Line 7: The `group_by()` function is applied again, but this time only by `lon` and `lat`. This modifies the grouping structure to remove the month grouping while retaining spatial grouping.
- Line 8: `The mutate()` function adds a new column, `num`, which assigns a sequence of numbers (`1:length(temp)`) to the grouped data. This effectively creates an index for each record within each longitude-latitude group.
- Line 9: The `ungroup()` function removes all grouping, ensuring that further operations on `monthlyData` are performed on the entire dataset rather than within groups.

Question 3

What is 'Occam's Razor'?

[5 marks]

Answer

Occam's Razor is sometimes attributed to the 14th-century philosopher William of Ockham, is a principle of parsimony that states: "Entities should not be multiplied beyond necessity." It is relevant to the BCB744 module because the principle of Occam's Razor is often interpreted as "the simplest explanation that sufficiently explains the data should be preferred over more complex alternatives." This is a nice guiding principle which might be useful in your research, especially when you are faced with multiple explanations for a phenomenon. The principle suggests that

the simplest explanation is often the best one, and that more complex explanations should only be considered when the simpler ones fail to account for the data. But, keep in mind that biological systems tend to be complex, and oversimplifying an explanation may ignore important interactions or heterogeneities.

Question 4

Explain the difference between R and RStudio.

[5 marks]

Answer

Taken verbatim from Tangled Bank:

R is a programming language and software environment for statistical computing and graphics. It provides a wide variety of statistical (linear and non-linear modelling, classical statistical tests, time-series analysis, classification, clustering, multivariate analyses, neural networks, and so forth) and graphical techniques, and is highly extensible.

RStudio is an integrated development environment (IDE) for R. It provides a graphical user interface (GUI) for working with R, making it easier to use for those who are less familiar with command-line interfaces. Some of the features provided by RStudio include:

- a code editor with syntax highlighting and code completion;
- a console for running R code;
- a graphical interface for managing packages and libraries;
- an integrated tools for plotting and visualisation; and
- support for version control with Git and SVN.

R is the core software for statistical computing, like a car's engine, while RStudio provides a more user-friendly interface for working with R, like the car's body, the seats, steering wheel, and other bells and whistles.

Question 5

By way of example, please explain some key aspects of R code conventions. For each line of code, explain also in English what aspects of the code are being adhered to.

For example:

1. `a <- b` is not the same as `a < -b`. The former is correct because there is a space preceding and following the assignment operator (`<-`, a less-than sign immediately followed by a dash to form an arrow); this has a different meaning from the latter, which is incorrect because there is no space between the less-than sign and the dash, reading as "a is less than negative b".

Hint: In your Word document, use a fixed-width font to indicate the code as a separate block which is distinct from the rest of the text.

[10 marks]

Answer

1. Proper use of indentation:

```
if (x > 0) {
  print("Positive number")
}
```

2. Use of meaningful variable names:

```
temperature <- 25
```

3. Use of comments to explain code:

```
# Calculate the mean temperature
mean_temp <- mean(temperature)
```

4. Consistent use of spacing around operators:

```
a <- b + c
```

5. Consistent use of compound object names:

A principles of writing clean and readable R code (or *any* code) is maintaining consistent variable naming conventions throughout a script or project. Mixing different naming styles – such as "snake_case" (words separated by underscores) and "camelCase" (capitalising the first letter of each subsequent word) – makes the code harder to read, maintain, and debug.

Examples:

```
# Example of consistent use of either convention:
my_variable <- 10 # snake case
another_variable <- 20 # camel case

# An example of inconsistent use of conventions:
myVariable <- 30 # camel case
yet_another_variable <- 40 # snake case

# This is also incorrect:
variable_one <- 13 # llowercase "one"
variable_Two <- 13 * 2 # uppercase "Two"
```

6. Avoiding the = as Assignment Operator

```
# Correct:
a <- 1
```

```
# Incorrect:
a = 1
```

7. Consistent use of spaces around # symbols in comments:

```
# This is correct:

# This is a comment
# This is another comment
# And another

# This is incorrect:

#This is a comment
# A comment?
#  Another comment
```

8. Correct use of `+` or `-` for unary operators:

```
# Correct:
a <- -b
```

9. Use of `TRUE` and `FALSE` instead of `T` and `F`:

```
# Correct:
is_positive <- TRUE

# Incorrect:
is_positive <- T
```

For more, refer to the tidyverse style guide.

Question 6

a. Explain why one typically prefers working with CSV files over Excel files in R.
b. What are the properties of a CSV file that make it more suitable for data analysis in R?
c. What are the properties of an Excel file that make it less suitable for data analysis in R?

[15 marks]

Answer

a)

CSV (Comma-Separated Values) files are preferred over Excel files due to their simplicity, compatibility, and efficiency in handling data. CSV files are stored as plain text, making them easy to read

and write across different software and platforms. They do not contain proprietary formatting, formulas, or metadata, which minimises the risk of unintended data transformations.

Excel files (.xls, .xlsx) are proprietary and designed for spreadsheet applications, incorporating complex formatting, formulas, and visual formatting that can interfere with data processing in R. Unlike CSV files, which can be directly read using base R functions like `read.csv()`, Excel files require additional packages such as **readxl** for data extraction. Excel's tendency to automatically modify data types – such as converting text to dates or numbers – is annoying and introduces errors, making CSV a more reliable format for reproducible data analysis.

b)

- CSV files store data in a simple text-based format that ensures easy readability by both humans and computers.
- Each row represents a single record, and fields are separated by commas (or another delimiter) to ensure a consistent tabular format.
- CSV files can be opened and edited using a wide range of software, including text editors, spreadsheets (e.g., Excel, Google Sheets), and statistical tools (e.g., R, Python).
- R provides optimised functions like `read.csv()` (base R) and `read_csv()` (tidyverse) for quickly reading CSV files without additional dependencies.
- Unlike Excel, CSV files do not contain embedded formulas, formatting, figures, or macros and these properties reduce the risk of unintended data stuff-ups.
- Being plain text, CSV files are typically smaller in size compared to Excel files.

c)

- Excel files are stored in a format (.xls, .xlsx) that is specific to Microsoft Excel; special packages (e.g., **readxl**, **openxlsx**) are needed to read them in R.
- Excel often automatically formats data and changes numeric values to dates or rounding decimal values. This can lead to errors in data analysis.
- Excel files support formulas, pivot tables, conditional formatting, and visual elements that may not be relevant for raw data processing in R.
- Users can store multiple sheets within a single Excel file and this makes it trickier to maintain a standardised structure when importing data into R.
- Excel files are not made for handling large datasets. Excel becomes very slow and is prone to crashing or memory limitations when dealing with 'big' data.
- Excel's binary files do not work with version control systems like Git.
- Excel files are complex and more prone to accidental modifications or corruption.

Question 7

Explain each of the following in the context of their use in R. For each, provide an example of how you would construct them in R:

a. A vector
b. A matrix
c. A dataframe

d. A list

Hint: See my hint under Question 5.

[20 marks]

Answer

(a) A vector in R is the simplest and most fundamental data structure. It is a one-dimensional collection of elements, all of the same type (e.g., numeric, character, or logical). Vectors can be created using the `c()` function. For example:

```r
# Creating a numeric vector
numbers <- c(1, 2, 3, 4, 5)

# Creating a character vector
names <- c("Acacia", "Protea", "Leucadendron")

# Creating a logical vector
logical_values <- c(TRUE, FALSE, TRUE)
```

(b) A matrix is a two-dimensional data structure where all elements must be of the same type. It is essentially an extension of a vector with a specified number of rows and columns.

```r
# Creating a matrix with 3 rows and 2 columns
my_matrix <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 3, ncol = 2)
```

(c) A dataframe is a two-dimensional data structure that can contain different data types in different columns (variables). It is the most commonly used data structure for data analysis in R and resembles a table with rows and columns.

```r
# Creating a dataframe
my_dataframe <- data.frame(
  Name = c("Acacia", "Protea", "Leucadendron"),
  Age = c(25, 30, 22),
  Height = c(85.5, 90.3, 78.0)
)
```

(d) A list is a flexible data structure that can store elements of different types, including vectors, matrices, dataframes, and even other lists. Unlike vectors and matrices, which require uniform data types, lists can contain heterogeneous elements.

```r
# Creating a list with different data types
# Uses the data created abobe, for example
my_list <- list(
```

```
plants = my_dataframe,
some_numbers = mu_matrix,
other_numbers = numbers
)
```

Question 8

a. Write a 150 to 200 word abstract about your Honours research project. In your abstract, draw attention to the types of data you will be expected to generate, and mention how these will be used to address your research question.

b. Explain which of the R data classes will be most useful in your research and why.

c. With reference to the abstract you wrote in Question 8.a, explain how you would visualise (or display your finding in tabular format) your research findings. Provide an example of how you would do this in R. Which of your research questions would be best answered using a visualisations or tables? What do you expect your visualisations or tables to show?

d. Provide an example of how you would create a plot or table in R. Generate mock code (it does not need to run) that you would use to create the plot or table.

Note 1: In the unlikely event that your research will not require visualisations or tables, please explain why this is the case and how you would communicate your findings.

Note 2: If you haven't defined your research project yet, describe a hypothetical project in your field of interest.

[30 marks]

Answer

This will have to be assessed based on the information quality produced in each abstract. Assign marks as follows:

a. Abstract: 30%
b. Data classes: 10%
c. Visualisation: 30%
d. Mock code: 30%

TOTAL MARKS: 110

Practical test

This is the open book assessment.

Below is a set of scripting problems to solve. You have 21 hours from the end of the Theory Test to complete this section Please write your code in an R script file and submit it to the iKamva platform by no later than 8:30 on Tuesday, 18 March 2025.

Please follow a clear structure (appropriate, clearly numbered headings and subheadings) in your code, including comments and explanations.

Ensure that all code runs without errors before submitting it – serious penalties will apply to non-functional scripts.

Naming convention: `Intro_R_Test_Practical_YourSurname.R`

Question 1

Download the `fertiliser_crop_data.csv` data.

The data represent an experiment designed to test whether or not fertiliser type and the density of planting have an effect on the yield of wheat. The dataset contains the following variables:

- Final yield (kg per acre) – make sure to convert this to the most suitable SI unit before continuing with your analysis
- Type of fertiliser (fertiliser type A, B, or C)
- Planting density (1 = low density, 2 = high density)
- Block in the field (north, east, south, west)

Undertake a full visual assessment of the dataset and establish which of the influential variables are most likely to have an effect on crop yield. Provide a detailed explanation of your findings.

[25 marks]

Answer

First, ensure the data are converted to SI units (e.g., kg per hectare) for consistency. I'd examine the tops and bottoms of the data with `head()` and `tail()` to see what we are dealing with, and it's also useful to do a `summary()`. I'd also print a table to see how the various measurements are distributed across the predictor variables.

```
library(tidyverse)
library(ggpubr)
fert <- read.csv(here::here("data", "BCB744", "fertiliser_crop_data.csv"))

# Convert to SI units
fert <- fert |>
  mutate(mass = mass / 0.40468564224)

# Convert acre to ha
fert <- fert |>
  mutate(mass = mass * 2.47105)

# Check the data
head(fert)
```

```
  density block fertilizer     mass
1       1 north          A 29451.95
```

```
2       2  east          A 29505.35
3       1 south          A 29315.65
4       2  west          A 29530.88
5       1 north          A 29434.80
6       2  east          A 29377.11
```

```
tail(fert)
```

```
   density block fertilizer     mass
91       1 south          C 29445.18
92       2  west          C 29481.30
93       1 north          C 29603.67
94       2  east          C 29532.04
95       1 south          C 29528.16
96       2  west          C 29433.59
```

```
summary(fert)
```

```
    density          block        fertilizer      mass
 Min.   :1.0   Length   :96   Length   :96   Min.   :29142
 1st Qu.:1.0   N.unique : 4   N.unique : 3   1st Qu.:29326
 Median :1.5   N.blank  : 0   N.blank  : 0   Median :29424
 Mean   :1.5   Min.nchar: 4   Min.nchar: 1   Mean   :29417
 3rd Qu.:2.0   Max.nchar: 5   Max.nchar: 1   3rd Qu.:29480
 Max.   :2.0                                 Max.   :29756
```

```
# Create a table showing the grouping structure of block, density, and fertiliser
fert %>%
  group_by(block, density, fertilizer) |>
  summarise(n = n()) |>
  head(12)
```

```
# A tibble: 12 × 4
# Groups:   block, density [4]
   block density fertilizer     n
   <chr>   <int> <chr>      <int>
 1 east        2 A              8
 2 east        2 B              8
 3 east        2 C              8
 4 north       1 A              8
```

```
 5 north       1 B             8
 6 north       1 C             8
 7 south       1 A             8
 8 south       1 B             8
 9 south       1 C             8
10 west        2 A             8
11 west        2 B             8
12 west        2 C             8
```

Then, I'd calculate the mean and standard deviation of the outcome variable for each level of the independent variables. This will give me a sense of the central tendency and spread of the data.

```
# Let's see group differences
# I also calculate the mean +/- SD here
fert |>
  group_by(density) |>
  summarise(mean = mean(mass),
            SD = sd(mass))
```

```
# A tibble: 2 × 3
  density   mean    SD
    <int>  <dbl> <dbl>
1       1 29378.  101.
2       2 29455.  107.
```

```
fert |>
  group_by(block) |>
  summarise(mean = mean(mass),
            SD = sd(mass))
```

```
# A tibble: 4 × 3
  block   mean    SD
  <chr>  <dbl> <dbl>
1 east  29467. 107.
2 north 29390. 104.
3 south 29366.  98.2
4 west  29443. 108.
```

```
fert |>
  group_by(fertilizer) |>
```

```
  summarise(mean = mean(mass),
            SD = sd(mass))
```

```
# A tibble: 3 × 3
  fertilizer   mean    SD
  <chr>       <dbl> <dbl>
1 A          29374. 114.
2 B          29403.  95.4
3 C          29473.  99.6
```

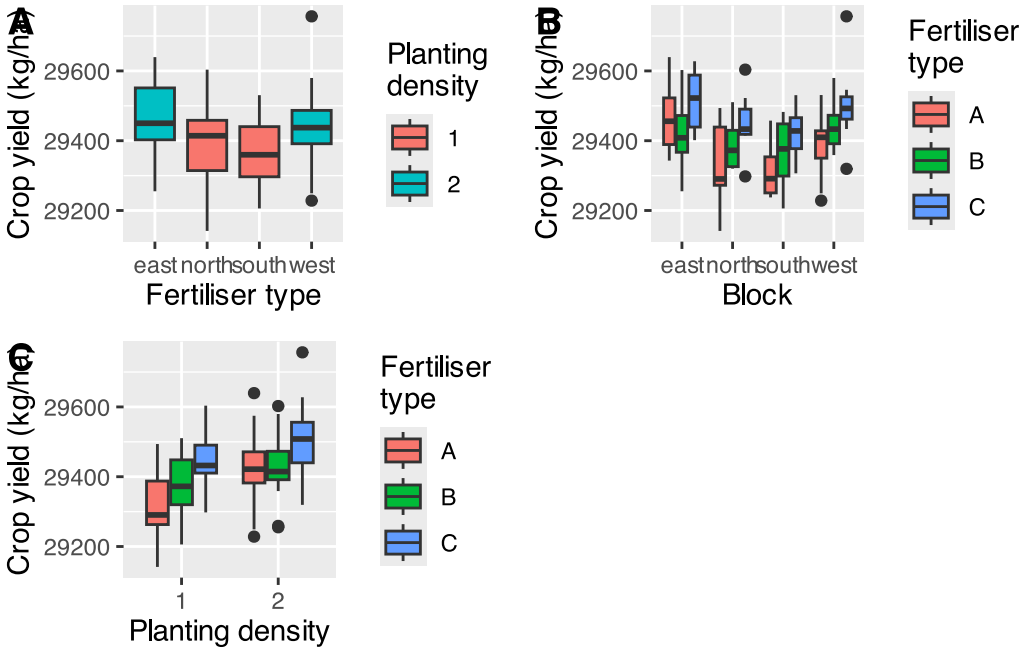
Next, I'd create plots of the data showing each level of independent variables that the outcome can vary over. Boxplots and barplots are the most appropriate, together with some form of variance indication (SE, SD, or CI).

```
# Visual assessment
# Create a boxplot of mass by density, fertiliser type, and density
# ... by fertiliser
plt1 <- fert |>
  ggplot(aes(x = block, y = mass, fill = as.factor(density))) +
  geom_boxplot(notch = FALSE) +
  labs(x = "Fertiliser type",
       y = "Crop yield (kg/ha)",
       fill = "Planting\ndensity")

# ... by density()
plt2 <- fert |>
  ggplot(aes(x = block, y = mass, fill = fertilizer)) +
  geom_boxplot(notch = FALSE) +
  labs(x = "Block",
       y = "Crop yield (kg/ha)",
       fill = "Fertiliser\ntype")

plt3 <- fert |>
  ggplot(aes(x = as.factor(density), y = mass, fill = fertilizer)) +
  geom_boxplot(notch = FALSE) +
  labs(x = "Planting density",
       y = "Crop yield (kg/ha)",
       fill = "Fertiliser\ntype")

# Arrange the plots
ggarrange(plt1, plt2, plt3, ncol = 2, nrow = 2, labels = "AUTO")
```



I'd then interpret these results, considering on the inferences I can make from visual assessments of the mean (or median) and the figures. The analyses must take into account all the influential variables: density, block, and fertiliser. Since this is Intro R and not Biostatistics, inferential stats tests aren't expected.

- I'd note that the mass of crop produced by fertiliser C is the greatest compared to both A and B; the effect of fertiliser B is no different (at least not consistently) than that of A. This response is seen if viewed across the different blocks and densities.
- The second planting density also yields a greater mass per ha, but it is also confounded with the block, so I'd need to consider this in my interpretation… East and west blocks have the highest yield, but they also were planted at a higher density to start with. To circumvent this problem, maybe calculate something like a yield per plant, a yield per unit area, or even relative growth rate, and then compare these across the different blocks and densities.
- These interpretations can be reached from examining the boxplots (or barplots) and the median (or mean), and some measure of variance such as ±SD (or ±CI).

For some bonus marks, I'd also consider the limitations of the study and potential confounding variables that may have influenced the results.

- I mentioned the confounding of block with density, but I'd also consider other factors that could have influenced the results, such as soil quality, weather conditions, or other unmeasured variables. None of these are mentioned, so the student can draw attention to this as unknowns that could affect the outcome.

Question 2

The Bullfrog Occupancy and Common Reed Invasion data are here: `AICcmodavg::bullfrog` (i.e. the `bullfrogs` dataset resides within the **AICcmodavg** package, which you might have to install).

Create a tidy dataframe from the bullfrog data.

[10 marks]

Answer

```
library(AICcmodavg)
data(bullfrog)

# View the first/last few rows of the dataset
head(bullfrog)
```

```
                     Location Reed.presence V1 V2 V3 V4 V5 V6 V7    Effort1
1                  Arbo_Mc_gill             0  0  0  0  0  0  0  0  1.3342857
2             Beauharnois_bassin             1  0  0  0  0  0  0  0 -0.6657143
3             Beauharnois_chemin             1  0  0  0  0  0  0  0 -0.6657143
4           Bois_de_liesse_elec              1  0 NA NA  0 NA NA  0 -4.6657143
5          Bois_de_liesse_grand              0  0  0  0  0  0  0  0  2.3342857
6 IBoucherville_chenal_a_pinard              1  0  0  0  1  0  0  0  0.3342857
     Effort2    Effort3    Effort4    Effort5    Effort6    Effort7 Type1 Type2
1  1.3342857  1.3342857  1.3342857  0.3342857  0.3342857  1.3342857     0     1
2 -0.6657143 -0.6657143 -0.6657143 -0.6657143 -0.6657143 -0.6657143     0     1
3 -0.6657143 -0.6657143 -0.6657143 -0.6657143 -0.6657143 -0.6657143     0     1
4 -8.6657143 -8.6657143 -4.6657143 -8.6657143 -8.6657143 -4.6657143     0     1
5  2.3342857  2.3342857  2.3342857  2.3342857  2.3342857  2.3342857     0     1
6  0.3342857  0.3342857  0.3342857  0.3342857  0.3342857  0.3342857     0     1
  Type3 Type4 Type5 Type6 Type7
1     1     0     1     1     0
2     1     0     1     1     0
3     1     0     1     1     0
4     1     0     1     1     0
5     1     0     1     1     0
6     1     0     1     1     0
```

```
# Convert the data to a tidy dataframe
# place all of the variables `V1` through `V7` under a single column
# that represents the survey occasion
# place all the variables `Effort1` through `Effort7` under a single
# column that represents the sampling effort
```

```
# place all the variables `Type1` through `Type7` under a single
# column that represents the survey type

# Reshape the data
tidy_bullfrog <- bullfrog |>
  pivot_longer(cols = starts_with("V"),
               names_to = "Occasion",
               values_to = "Occasion.val") |>
  pivot_longer(cols = starts_with("Effort"),
               names_to = "Effort",
               values_to = "Effort.val") |>
  pivot_longer(cols = starts_with("Type"),
               names_to = "Type",
               values_to = "Type.val")

# View the first/last few rows of the tidy dataframe
head(tidy_bullfrog)
```

```
# A tibble: 6 × 8
  Location  Reed.presence Occasion Occasion.val Effort Effort.val Type  Type.val
  <fct>             <int> <chr>           <int> <chr>       <dbl> <chr>    <int>
1 Arbo_Mc_…             0 V1                  0 Effor…       1.33 Type1        0
2 Arbo_Mc_…             0 V1                  0 Effor…       1.33 Type2        1
3 Arbo_Mc_…             0 V1                  0 Effor…       1.33 Type3        1
4 Arbo_Mc_…             0 V1                  0 Effor…       1.33 Type4        0
5 Arbo_Mc_…             0 V1                  0 Effor…       1.33 Type5        1
6 Arbo_Mc_…             0 V1                  0 Effor…       1.33 Type6        1
```

To consider in marking the answer:

- The student should have reshaped the data into a tidy format, with each row representing a unique observation.
- The student should have correctly identified the variables to be reshaped and the new column names.
- The student should have demonstrated an understanding of the `pivot_longer()` (or equivalent) function and its arguments.
- They should have applied an consistent naming convention for the new columns.

Question 3

The Growth Curves for Sitka Spruce Trees in 1988 and 1989 data are here: `MASS::Sitka` and `MASS::Sitka89`.

Combine the two datasets and provide an analysis of the growth curves for Sitka spruce trees in 1988 and 1989. Give graphical support for the idea that i) ozone affects the growth of Sitka

spruce trees, and ii) the growth of Sitka spruce trees is affected by the year of measurement. In addition to showing the overall response in each year x treatment, also ensure that the among tree variability is visible.

Explain your findings.

[20 marks]

Answer

```
# load data
sitka <- MASS::Sitka
sitka89 <- MASS::Sitka89

# Look at them
head(sitka)
```

```
  size Time tree treat
1 4.51  152    1 ozone
2 4.98  174    1 ozone
3 5.41  201    1 ozone
4 5.90  227    1 ozone
5 6.15  258    1 ozone
6 4.24  152    2 ozone
```

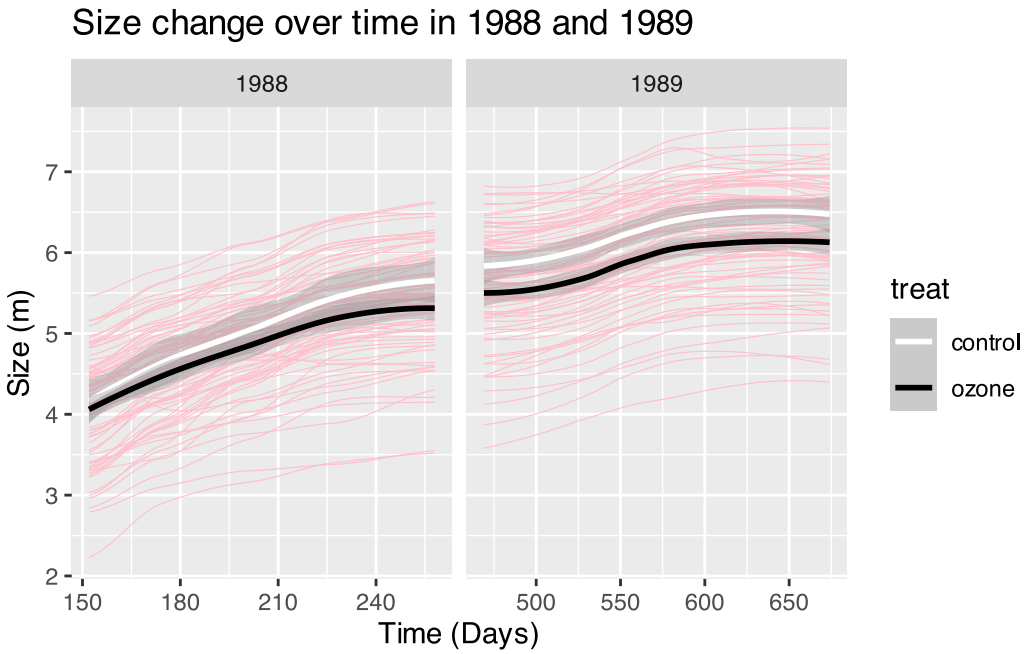
```
head(sitka89)
```

```
  size Time tree treat
1 6.16  469    1 ozone
2 6.18  496    1 ozone
3 6.48  528    1 ozone
4 6.65  556    1 ozone
5 6.87  579    1 ozone
6 6.95  613    1 ozone
```

```
# Combine the two datasets
# rbind them, and create a new column for the year (1988 for `Sitka` and
# 1989 for `Sitka89`)
sitka$year <- 1988
sitka89$year <- 1989

# Combine the datasets
sitka_combined <- rbind(sitka, sitka89)
```

```r
# Make some plots
ggplot(data = sitka_combined, aes(x = Time, y = size)) +
  geom_smooth(aes(group = tree),
              color = "pink",  # among-tree variability lines
              linewidth = 0.2,
              se = FALSE) +
  geom_smooth(aes(color = treat),  # overall lines mapped to treatment
              linewidth = 1,
              se = TRUE) +
  scale_color_manual(values = c("control" = "white", "ozone" = "black")) +
  labs(x = "Time (Days)",
       y = "Size (m)",
       title = "Size change over time in 1988 and 1989") +
  facet_wrap(~ year, scales = "free_x")
```



The figure shows the growth curves for Sitka spruce trees in 1988 and 1989. The pink lines represent the growth curves for individual trees, while the red and blie lines represent the average growth curves for the control and ozone-treated trees, respectively. The growth curves for the ozone-treated trees appear to be lower than those for the control trees, indicating that ozone affects the growth of Sitka spruce trees. Additionally, although the trees are all taller in 1989, the growth curves for 1989 are generally lower (i.e. their rate of change over time) than those for 1988, suggesting that the growth of Sitka spruce trees is affected by the year of measurement. This suggests the trees are maturing and their growth rates are slowing down. The variability among

trees is also visible and very substantial, with some trees being bigger in 1988 compared to some in 1989.

Question 4

The Frog Dehydration Experiment on Three Substrate Types data can be accessed here: `AICcmodavg::dry.frog`.

a. Based on the dataset, what do you think was the purpose of this study? Provide a 200 word synopsis as your answer.
b. Create new columns in the dataframe showing:
   - the final mass;
   - the percent mass lost; and
   - the percent mass lost as a function of the initial mass of each frog.
c. Provide the R code that would have resulted in the data in the variables `cent_Initial_mass` and `cent_Air`.
d. An analysis of the factors responsible for dehydration rates in frogs. In your analysis, consider the effects substrate type, initial mass, air temperature, and wind.
e. Provide a brief discussion of your findings.

[25 marks]

Answer

a. The investigators sought to determine whether anthropogenic disturbances that remove ground cover or alter substrate moisture impede the dispersal and homing capacities of frog populations. They hypothesised that desiccation risks, heightened predation exposure, and substrate temperature extremes could collectively inhibit anuran relocations across open landscapes. By scrutinising individual orientation behaviour and homing success, they hoped to elucidate whether frogs could detect distant patches of suitable habitat and whether traversing hostile terrain diminished the likelihood of reaching them. They also questioned if contrasting body sizes, reflecting differing surface-to-volume ratios, influenced dehydration and survival patterns during overland migrations. Seeking mechanistic clarity rather than mere distributional data, they devised a series of field-based translocation tests, effectively isolating frogs on disturbed or undisturbed surfaces to compare path selection, movement propensity, and ultimate reunion with their original ponds. Additionally, by examining dehydration rates and postural adaptations on varied substrates – ranging from vegetation-rich bogs to dry peat fields – they aimed to quantify physiological constraints that shape dispersal outcomes. Through these interconnected experiments, they intended to shed further light on the dynamic interplay between environmental structure, amphibian behaviour, and survival to inform predictive models of habitat connectivity and population resilience in areas undergoing increasingly disruptive land-use transformations.

b. To calculate the final mass, percent mass lost, and percent mass lost as a function of the initial mass of each frog, we can use the following code:

```
library(AICcmodavg)
data(dry.frog)

# Looking the data
head(dry.frog)
```

```
  Individual Species Shade  SVL Substrate Initial_mass Mass_lost Airtemp
1          1   Racla     0 7.27      SOIL         38.5       8.3      31
2          2   Racla     0 7.00  SPHAGNUM         31.0       3.6      31
3          3   Racla     0 6.83      PEAT         23.6       4.7      31
4          4   Racla     0 7.26      PEAT         37.4       7.0      22
5          5   Racla     0 7.43      SOIL         44.4       7.7      22
6          6   Racla     0 5.75  SPHAGNUM         16.4       1.6      22
  Wind_cat Cloud cent_Initial_mass Initial_mass2 cent_Air Perc.cloud Wind
1        3    20         20.361157    414.576715  2.64876       0.20    1
2        3    20         12.861157    165.409360  2.64876       0.20    1
3        3    20          5.461157     29.824236  2.64876       0.20    1
4        2     5         19.261157    370.992170 -6.35124       0.05    1
5        2     5         26.261157    689.648368 -6.35124       0.05    1
6        2     5         -1.738843      3.023575 -6.35124       0.05    1
  log_Mass_lost
1      3.217231
2      2.201634
3      2.510962
4      3.000000
5      3.121015
6      1.378512
```

```
# Create new columns
frog <- dry.frog |>
  mutate(Final_mass = (Initial_mass - Mass_lost)) |>
  mutate(Perc_mass_loss = (Initial_mass / Final_mass) * 100) |>
  mutate(Perc_mass_loss_initial = Perc_mass_loss)
```

c. Here, 'centered' means to subtract the overall mean from each value. The R code that would have resulted in the data in the variables `cent_Initial_mass` and `cent_Air` is as follows:

```
# Create the cent_Initial_mass and cent_Air columns
frog <- frog |>
  mutate(cent_Initial_mass_new = (Initial_mass - mean(Initial_mass))) |>
  mutate(cent_Air_new = (Airtemp - mean(Airtemp)))
```

d. To create a purely visual analysis of the factors responsible for dehydration rates in frogs, we can trend lines, scatter plots, boxplots, etc. examine the effects of substrate type, initial mass, air temperature, and wind on the percent mass lost. The code would look something like this:

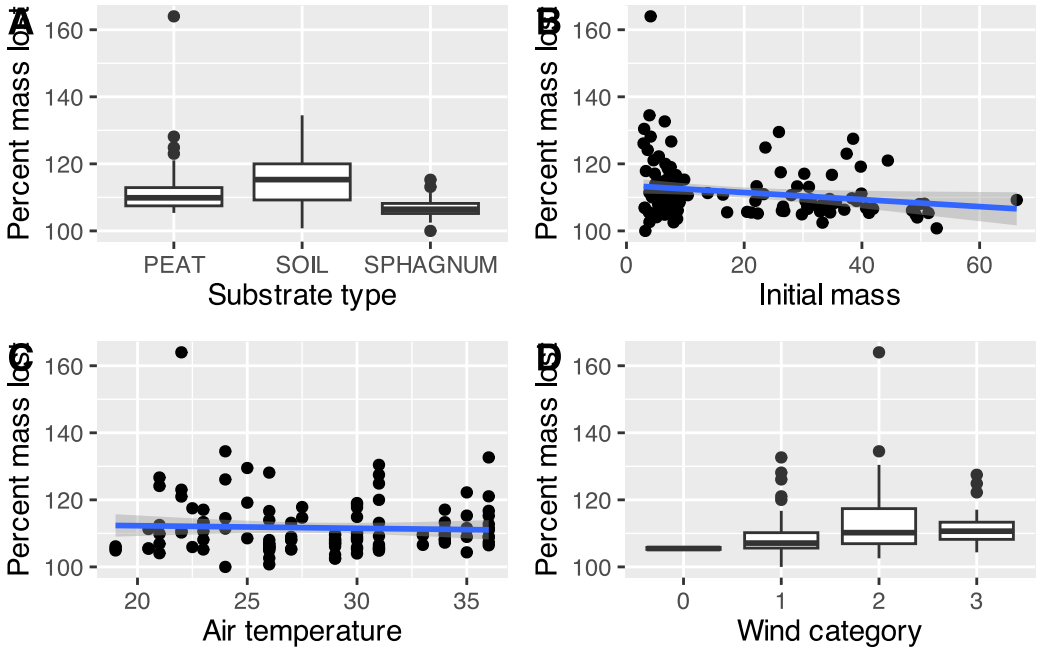
```
# Create graphs of all the influential variables and their effects
# on the mass loss:
# ... the effect of substrate
plt1 <- frog |>
  ggplot(aes(x = Substrate, y = Perc_mass_loss)) +
  geom_boxplot() +
  labs(x = "Substrate type",
       y = "Percent mass lost")

# ... the effect of initial mass
plt2 <- frog |>
  ggplot(aes(x = Initial_mass, y = Perc_mass_loss)) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(x = "Initial mass",
       y = "Percent mass lost")

# ... the effect of air temperature
plt3 <- frog |>
  ggplot(aes(x = Airtemp, y = Perc_mass_loss)) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(x = "Air temperature",
       y = "Percent mass lost")

# ... the effect of the wind category
plt4 <- frog |>
  ggplot(aes(x = as.factor(Wind_cat), y = Perc_mass_loss)) +
  geom_boxplot() +
  labs(x = "Wind category",
       y = "Percent mass lost")

ggarrange(plt1, plt2, plt3, plt4, ncol = 2, nrow = 2, labels = "AUTO")
```



e. Frogs on soil showed the largest overall percent-mass losses, while sphagnum moss minimised losses. Larger individuals tended to lose a lower fraction of their mass than smaller ones. Temperature had a weak negative (or no effect, statistically) trend, whereas stronger wind categories tended to correspond to elevated mass-loss percentages.

Question 5

Consider this script:

```
ggplot(points, aes(x = group, y = count)) +
  geom_boxplot(aes(colour = group), size = 1, outlier.colour = NA) +
  geom_point(position = position_jitter(width = 0.2), alpha = 0.3) +
  facet_grid(group ~ ., scales = "free") +
  labs(x = "", y = "Number of data points") +
  theme(legend.position = "none",
    strip.background = element_blank(),
    strip.text = element_blank())
```

a. Generate fictitious (random, normal) data that can be plotted using the code, above. Make sure to assemble these data into a dataframe suitable for plotting, complete with correct column titles.
b. Apply the script *exactly as stated* to the data to demonstate your understanding of the code and convince the examiner of your understanding of the correct data structure.

[10 marks]

Answer

a. Generate the data:

```
# Generate some fictitious data
set.seed(123)  # For reproducibility

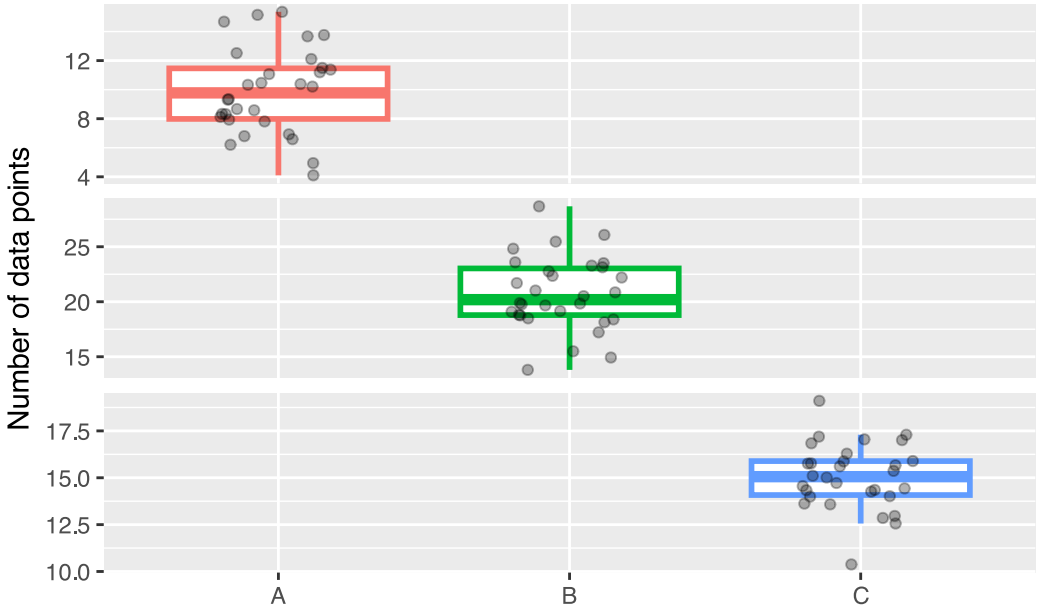
points <- data.frame(
  group = rep(c("A", "B", "C"), each = 30),
  count = c(
    rnorm(30, mean = 10, sd = 3),
    rnorm(30, mean = 20, sd = 4),
    rnorm(30, mean = 15, sd = 2)
  )
)

# Quick check
head(points)
```

```
  group     count
1     A  8.318573
2     A  9.309468
3     A 14.676125
4     A 10.211525
5     A 10.387863
6     A 15.145195
```

b. Apply the script to the data:

```
# Plot the data
ggplot(points, aes(x = group, y = count)) +
  geom_boxplot(aes(colour = group), size = 1, outlier.colour = NA) +
  geom_point(position = position_jitter(width = 0.2), alpha = 0.3) +
  facet_grid(group ~ ., scales = "free") +
  labs(x = "", y = "Number of data points") +
  theme(legend.position = "none",
    strip.background = element_blank(),
    strip.text = element_blank())
```



Question 6

For this assessment, you will analyse the built-in R dataset `datasets::UKDriverDeaths`, which contains monthly totals of car drivers killed or seriously injured in road accidents in Great Britain from January 1969 to December 1984. This time series data allows for examination of long-term trends, seasonal patterns, and potential correlations with societal factors.

a. **Data Exploration and Preparation**

i. Load the `UKDriverDeaths` dataset and examine its structure. Convert the time series data into a standard data frame format with separate columns for:
   - Year
   - Month (both as a number and as a factor with proper names)
   - Number of deaths/injuries

ii. Create a new variable that classifies each month into seasons (Winter: Dec-Feb, Spring: Mar-May, Summer: Jun-Aug, Autumn: Sep-Nov).

iii. Create another variable identifying whether each observation falls during a major energy crisis period (e.g., the oil crises of 1973-1974 and 1979-1980).

iv. Identify and handle any potential inconsistencies or issues in the dataset that might affect subsequent analyses.

[20 marks]

Answer

```r
# Load the dataset
data("UKDriverDeaths", package = "datasets")
```

```
# Examine the structure
str(UKDriverDeaths)
```

```
 Time-Series [1:192] from 1969 to 1985: 1687 1508 1507 1385 1632 ...
```

```
# Convert the time series data into a standard data frame format
df <- data.frame(
  Year = floor(time(UKDriverDeaths)),
  Month = month.abb[cycle(UKDriverDeaths)],
  Deaths = as.numeric(UKDriverDeaths)
)

head(df)
```

```
  Year Month Deaths
1 1969   Jan   1687
2 1969   Feb   1508
3 1969   Mar   1507
4 1969   Apr   1385
5 1969   May   1632
6 1969   Jun   1511
```

```
# Create a new variable for seasons
df <- df |>
  mutate(Season = case_when(
    Month %in% c("Dec", "Jan", "Feb") ~ "Winter",
    Month %in% c("Mar", "Apr", "May") ~ "Spring",
    Month %in% c("Jun", "Jul", "Aug") ~ "Summer",
    Month %in% c("Sep", "Oct", "Nov") ~ "Autumn"
  ))

# Create a variable for major energy crisis periods
df <- df |>
  mutate(Energy_Crisis = case_when(
    Year %in% 1973:1974 | Year %in% 1979:1980 ~ "Yes",
    TRUE ~ "No"
  ))

# Check for inconsistencies
head(df)
```

```
  Year Month Deaths Season Energy_Crisis
1 1969   Jan   1687 Winter            No
2 1969   Feb   1508 Winter            No
3 1969   Mar   1507 Spring            No
4 1969   Apr   1385 Spring            No
5 1969   May   1632 Spring            No
6 1969   Jun   1511 Summer            No
```

```
tail(df)
```

```
    Year Month Deaths Season Energy_Crisis
187 1984   Jul   1222 Summer            No
188 1984   Aug   1284 Summer            No
189 1984   Sep   1444 Autumn            No
190 1984   Oct   1575 Autumn            No
191 1984   Nov   1737 Autumn            No
192 1984   Dec   1763 Winter            No
```

```
summary(df)
```

```
      Year            Month         Deaths           Season      Energy_Crisis
 Min.   :1969   Length   :192   Min.   :1057   Length   :192   Length   :192
 1st Qu.:1973   N.unique : 12   1st Qu.:1462   N.unique :  4   N.unique :  2
 Median :1976   N.blank  :  0   Median :1631   N.blank  :  0   N.blank  :  0
 Mean   :1976   Min.nchar:  3   Mean   :1670   Min.nchar:  6   Min.nchar:  2
 3rd Qu.:1980   Max.nchar:  3   3rd Qu.:1851   Max.nchar:  6   Max.nchar:  3
 Max.   :1984                   Max.   :2654
```

b. **Temporal Trend Analysis**

i. Create a comprehensive visualisation showing the full time series with:
- Clear temporal trends
- A smoothed trend line
- Vertical lines or shading indicating major UK policy changes related to road safety (e.g., 1983 seat belt law)
- Annotations for key events

ii. Develop a visualisation examining monthly fatality averages across the entire period, ordered appropriately to show seasonal patterns.

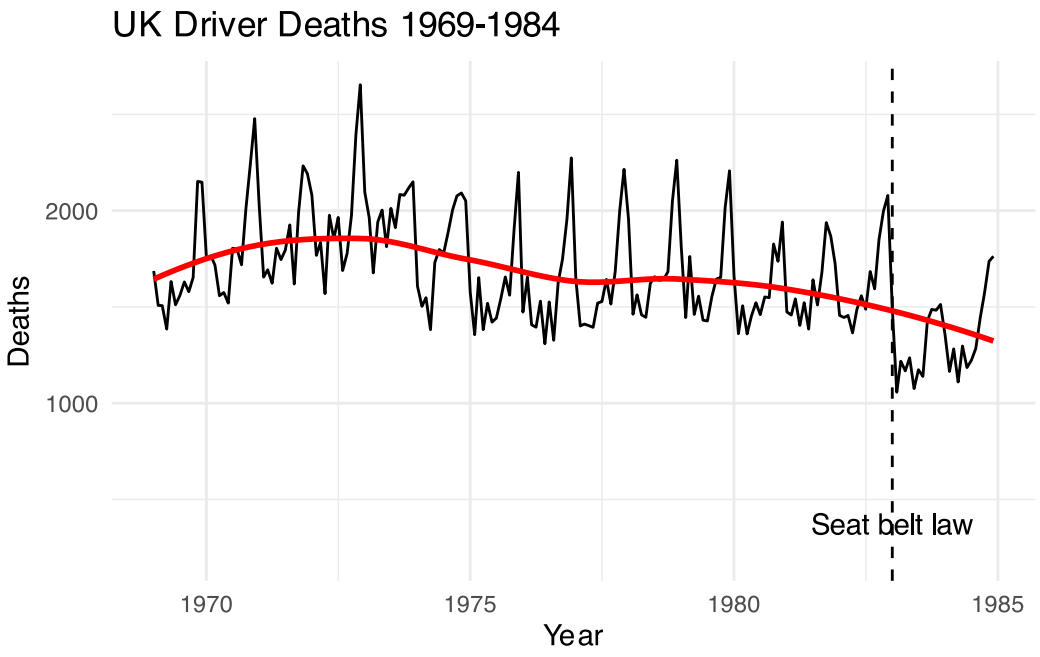
iii. Create a visualisation that compares annual patterns between the first half of the dataset (1969-1976) and the second half (1977-1984).

iv. Using *tidy* data manipulation techniques, calculate and visualise the year-over-year percent change in fatalities for each month throughout the dataset.

[20 marks]

Answer

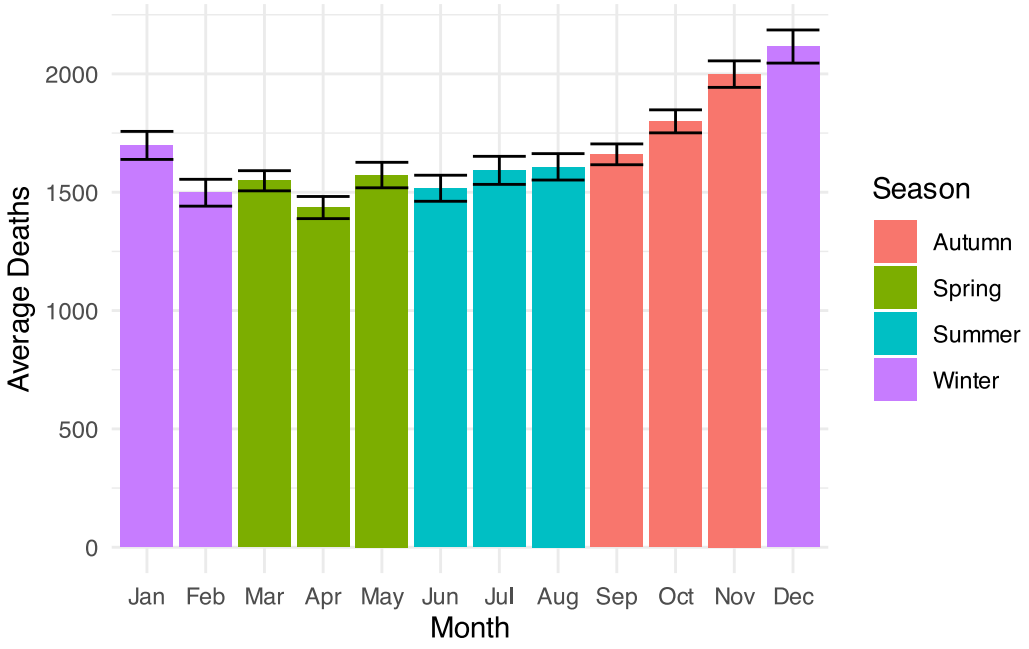
```
# i. Full time series visualisation
ggplot(df, aes(x = as.Date(paste(Year, Month, "01", sep = "-"),
                           format = "%Y-%b-%d"), y = Deaths)) +
  geom_line() +
  geom_smooth(method = "loess", se = FALSE, color = "red") +
  geom_vline(xintercept = as.Date(c("1983-01-01")), linetype = "dashed") +
  annotate("text", x = as.Date("1983-01-01"), y = 200, label = "Seat belt law",
vjust = -1) +
  labs(x = "Year", y = "Deaths", title = "UK Driver Deaths 1969-1984") +
  theme_minimal()
```




```
# ii. Monthly fatality averages
# ensure the months are ordered correctly and include also
# SD for each month (as determined across the years)
df$Month <- factor(df$Month, levels = month.abb)

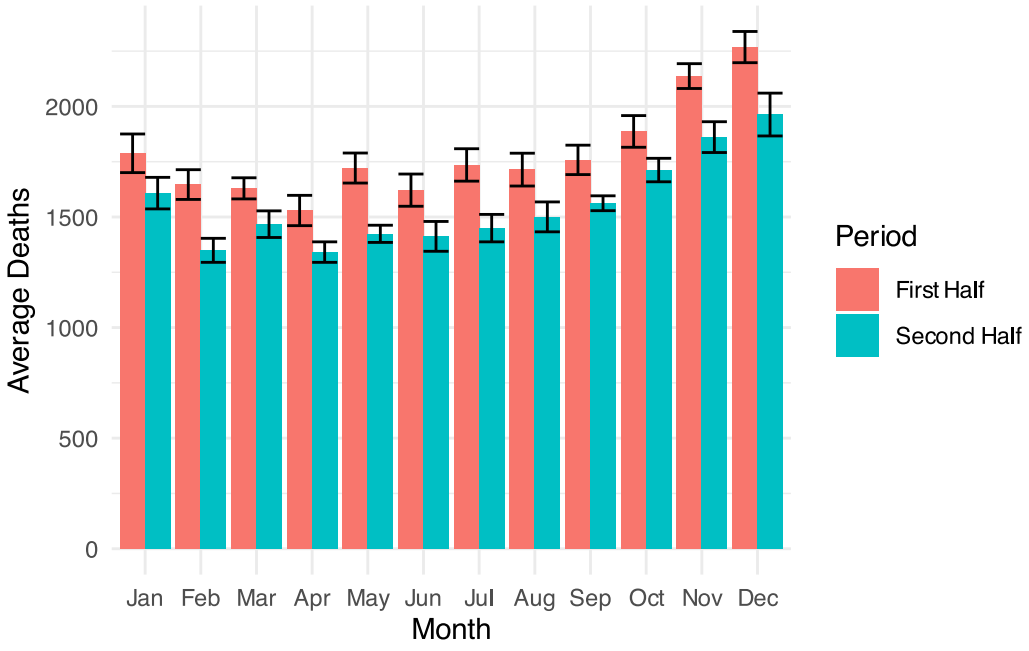
ggplot(df, aes(x = Month, y = Deaths, fill = Season)) +
  geom_bar(stat = "summary", fun = "mean", position = "dodge") +
```

```
  geom_errorbar(stat = "summary", fun.data = "mean_se", position = "dodge") +
  labs(x = "Month", y = "Average Deaths", fill = "Season") +
  theme_minimal()
```



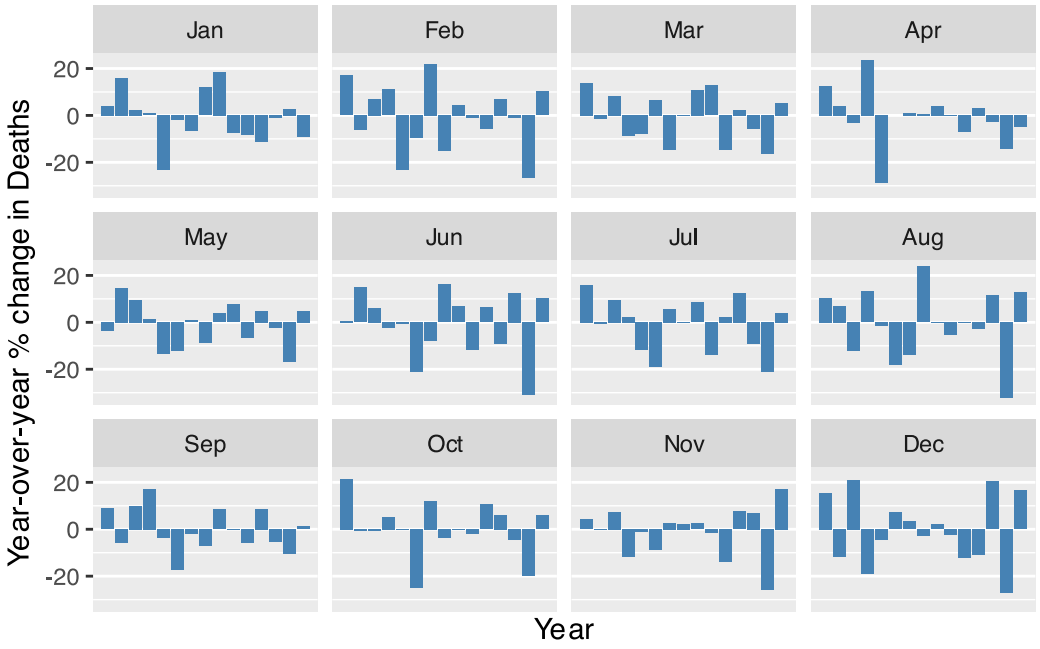

```
# iii. Annual patterns comparison
# ensure the years are ordered correctly and that the SDs are presented
df <- df |>
  mutate(Half = case_when(
    Year %in% 1969:1976 ~ "First Half",
    Year %in% 1977:1984 ~ "Second Half"
  ))

ggplot(df, aes(x = Month, y = Deaths, fill = Half)) +
  geom_bar(stat = "summary", fun = "mean", position = "dodge") +
  geom_errorbar(stat = "summary", fun.data = "mean_se", position = "dodge") +
  labs(x = "Month", y = "Average Deaths", fill = "Period") +
  theme_minimal()
```




```r
# iv. Year-over-year percent change
df_yoy <- df %>%
  arrange(Year, Month) %>%           # ensure rows are in ascending time order
  group_by(Month) %>%                # group so that we compare same months
  mutate(Deaths_Pct_Change =         # (current - previous) / previous * 100
    100 * (Deaths - lag(Deaths)) / lag(Deaths)
  ) %>%
  ungroup()

# Plot year-over-year % changes in a facetted bar chart
ggplot(df_yoy, aes(x = Year, y = Deaths_Pct_Change)) +
  geom_col(fill = "steelblue") +
  facet_wrap(~ Month, nrow = 3) +
  labs(
    x = "Year",
    y = "Year-over-year % change in Deaths"
  ) +
  theme(
    axis.text.x = element_text(angle = 90, vjust = 0.5)
  ) +
  scale_x_discrete(
    breaks = c("1969", "1974", "1979", "1984")  # Adjust as you wish
  )
```



c. **Pattern Analysis and Decomposition**

i. Calculate and visualise the average number of fatalities by season across all years.
ii. Create a heatmap showing fatalities by month and year, with appropriate color scaling to highlight temporal clusters.
iii. Implement a decomposition of the time series to separate: - The overall trend - Seasonal patterns - Remaining variation
iv. Visualise each component and discuss what factors might contribute to the patterns observed.

Note: Some of this will be new to you. But don't worry, use any means available to you to solve the problem.

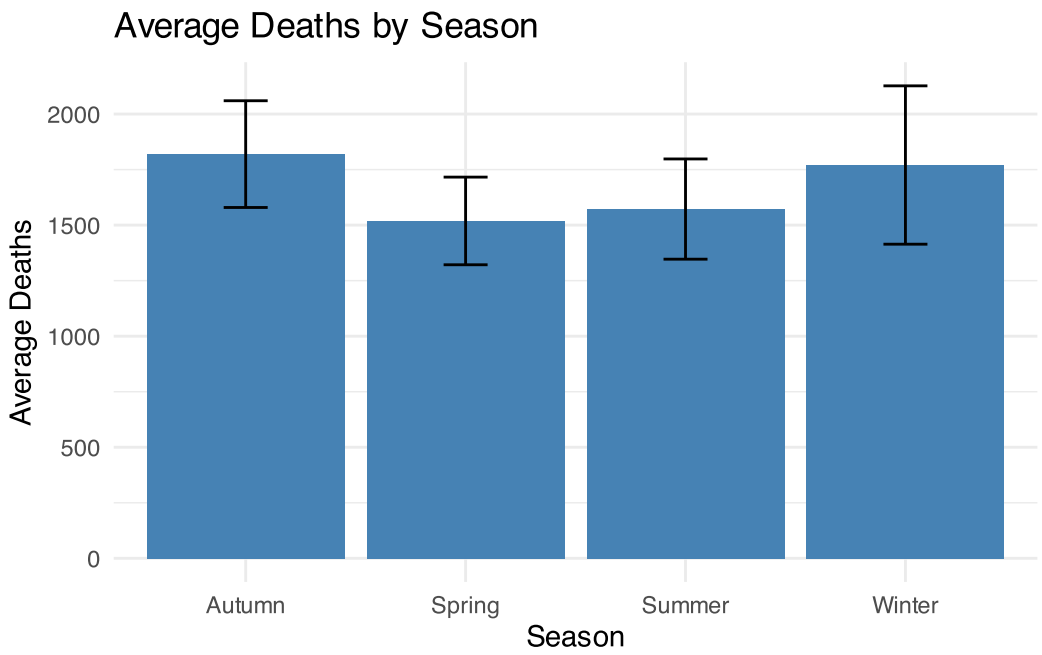
[25 marks]

Answer

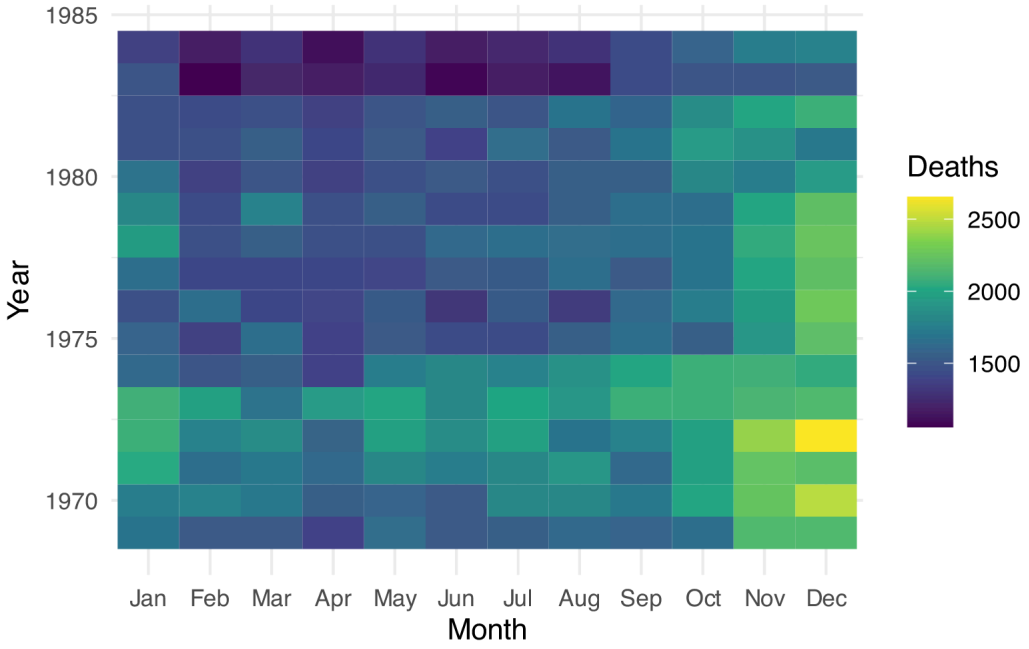
```
# i. Average fatalities by season: mean with SD
df_season <- df %>%
  group_by(Season) %>%
  summarise(
    Avg_Deaths = mean(Deaths),
    SD_Deaths  = sd(Deaths)
  )

ggplot(df_season, aes(x = Season, y = Avg_Deaths)) +
```

```
  geom_col(fill = "steelblue") +
  geom_errorbar(
    aes(
      ymin = Avg_Deaths - SD_Deaths,
      ymax = Avg_Deaths + SD_Deaths
    ),
    width = 0.2
  ) +
  labs(
    x = "Season",
    y = "Average Deaths",
    title = "Average Deaths by Season"
  ) +
  theme_minimal()
```



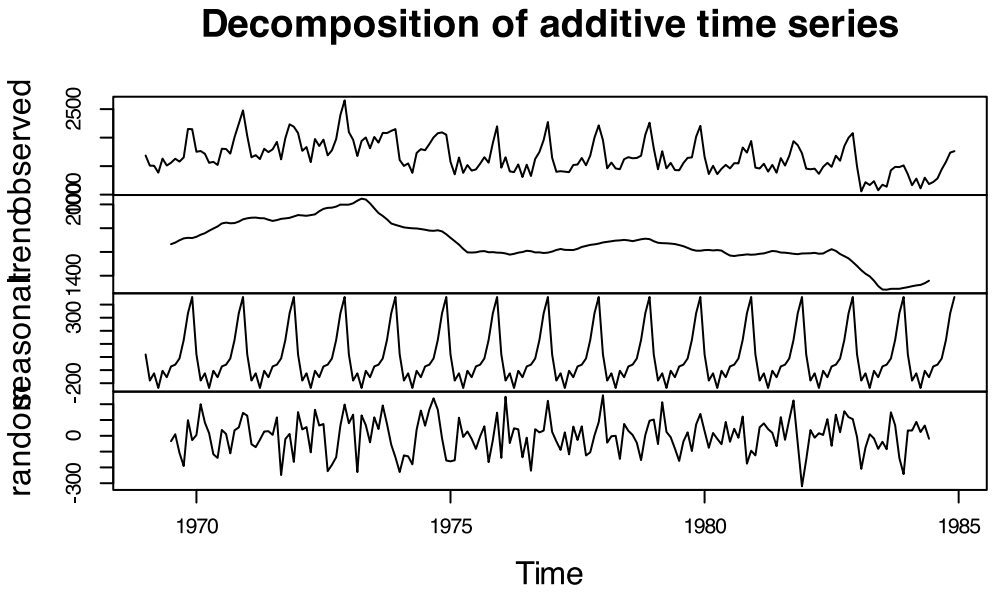

```
# ii. Heatmap of fatalities by month and year
ggplot(df, aes(x = Month, y = Year, fill = Deaths)) +
  geom_tile() +
  scale_fill_viridis_c() +
  labs(x = "Month", y = "Year", fill = "Deaths") +
  theme_minimal()
```




```
# iii. Time series decomposition
# Decompose the time series into trend, seasonal, and remainder components
# First, convert df to a time-series (ts) object in chronological order
df_ts <- df %>%
  arrange(Year, Month) %>%
  pull(Deaths) %>%
  ts(start = c(1969, 1), frequency = 12)

# Then, decompose using classical decomposition
ts_decomposed <- decompose(df_ts, type = "additive")

# iv. Plot the results
plot(ts_decomposed)
```



d. **Data manipulation**

Starting with the data as presented in the `UKDriverDeaths` dataset, create a new dataframe identical to the `Seatbelts` dataset.

[5 marks]

Answer

The two datasets differ substantially in structure and content. `Seatbelts` has columns for drivers, front passengers, rear passengers, # (etc.), while `UKDriverDeaths` only has a measurement variable for the total number of drivers regardless of whether or not they were killed or where in the vehicle they were, etc… So, we cannot perfectly recreate the `Seatbelts` dataset from the `UKDriverDeaths` dataset. However, we can create a column with the total drivers only…

```
# Look at the original `Seatbelts` dataset
data("Seatbelts", package = "datasets")
head(Seatbelts)
```

```
         DriversKilled drivers front rear   kms PetrolPrice VanKilled law
Jan 1969           107    1687   867  269  9059   0.1029718        12   0
Feb 1969            97    1508   825  265  7685   0.1023630         6   0
Mar 1969           102    1507   806  319  9963   0.1020625        12   0
Apr 1969            87    1385   814  407 10955   0.1008733         8   0
May 1969           119    1632   991  454 11823   0.1010197        10   0
Jun 1969           106    1511   945  427 12391   0.1005812        13   0
```

```r
data("UKDriverDeaths", package = "datasets")
UKDriverDeaths
```

```
      Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct  Nov  Dec
1969 1687 1508 1507 1385 1632 1511 1559 1630 1579 1653 2152 2148
1970 1752 1765 1717 1558 1575 1520 1805 1800 1719 2008 2242 2478
1971 2030 1655 1693 1623 1805 1746 1795 1926 1619 1992 2233 2192
1972 2080 1768 1835 1569 1976 1853 1965 1689 1778 1976 2397 2654
1973 2097 1963 1677 1941 2003 1813 2012 1912 2084 2080 2118 2150
1974 1608 1503 1548 1382 1731 1798 1779 1887 2004 2077 2092 2051
1975 1577 1356 1652 1382 1519 1421 1442 1543 1656 1561 1905 2199
1976 1473 1655 1407 1395 1530 1309 1526 1327 1627 1748 1958 2274
1977 1648 1401 1411 1403 1394 1520 1528 1643 1515 1685 2000 2215
1978 1956 1462 1563 1459 1446 1622 1657 1638 1643 1683 2050 2262
1979 1813 1445 1762 1461 1556 1431 1427 1554 1645 1653 2016 2207
1980 1665 1361 1506 1360 1453 1522 1460 1552 1548 1827 1737 1941
1981 1474 1458 1542 1404 1522 1385 1641 1510 1681 1938 1868 1726
1982 1456 1445 1456 1365 1487 1558 1488 1684 1594 1850 1998 2079
1983 1494 1057 1218 1168 1236 1076 1174 1139 1427 1487 1483 1513
1984 1357 1165 1282 1110 1297 1185 1222 1284 1444 1575 1737 1763
```

```r
# Create a new dataframe identical to the Seatbelts dataset
Seatbelts_v2 <- data.frame(
  drivers = UKDriverDeaths
)
head(Seatbelts_v2)
```

```
  drivers
1    1687
2    1508
3    1507
4    1385
5    1632
6    1511
```

TOTAL MARKS: 160

– THE END –

Bibliography